ZeoAuction: Distributed Real-Time Auction Platform

Web Services & Service-Oriented Architecture

M.Sc. IT (Semester 3) — Group Project (Team of 8)

August 2026

1. Project Title & Description

ZeoAuction is a microservices-based real-time auction platform built on Service-Oriented Architecture (SOA) principles. Users create, bid on, and settle auctions across independently deployable services that communicate through Apache Kafka (event-driven, asynchronous) and REST APIs (synchronous, request–response). Sellers must complete identity verification (KYC) before they may list; the platform charges a flat convenience fee on each completed sale; and settlement funds are held by the platform in escrow for a bounded buyer-protection window, released to the seller only when the window expires without a dispute or the buyer confirms receipt. The platform is designed to sustain high concurrent bid throughput with low end-to-end latency, enforce consistency across service boundaries using the Saga pattern, detect fraudulent bidding behaviour from a live event stream, and degrade gracefully under partial failure through circuit breakers and per-service health checks with automatic restart.

2. Application Type

Web application with a React 18 single-page frontend and Python (FastAPI) microservices on the backend. The architecture is hybrid: a documented REST API exposes the same capabilities to mobile clients and third-party integrators, while WebSocket channels carry live bid updates. Every backend service is containerised with Docker, runs locally under a single Docker Compose file, and deploys from the same image as an independent service on Render; the React frontend deploys to Vercel.

3. Problem Statement & Motivation

Limitations of monolithic, centralised auction systems:

- Write contention on a single hot row (the current auction price) throttles throughput as concurrency rises.
- Simultaneous equal bids produce ambiguous ordering, so the winner depends on client clocks or arbitrary database interleaving.
- Outbid notifications delivered by client-side polling arrive several seconds late, by which point the bidding window may have closed.
- Failure of one subsystem (e.g. the payment processor) takes the entire application offline.
- Fraud patterns such as shill bidding are found in overnight batch analysis, hours after the auction has already settled.
- Payment is released to the seller on capture, so a buyer who receives nothing, or receives an item that does not match the listing, has no recourse inside the platform.
- Unverified sellers can list anonymously, so a suspended account is recreated in minutes and there is no accountable identity behind a listing.
- Components cannot be scaled independently: load on payment processing degrades bid ingestion.

How ZeoAuction addresses each: authoritative server-side sequencing for deterministic conflict resolution; Redis-backed write buffering with batched persistence; WebSocket push instead of polling; per-service horizontal scaling; circuit breakers with defined fallback behaviour; a streaming fraud detector consuming the same Kafka topic as the bid pipeline, so detection happens in-flight rather than after the fact; platform-held escrow with a time-bounded dispute window backed by administrator adjudication; and a mandatory seller KYC gate enforced at the service layer, so listing is impossible without a verified identity.

4. Target Users & Key Use Cases

User classes: individual sellers (hobbyists, small traders), buyers and collectors, platform administrators (KYC review, moderation, dispute adjudication), and third-party applications consuming the public REST API.

Core use cases:

- A prospective seller registers and submits KYC documents. An administrator reviews and approves them. Only then does the listing endpoint accept a request from that account; an unverified seller receives an explicit rejection.
- A verified seller creates an auction with title, images, reserve price, and duration.
- A buyer places a bid; the system validates funds and eligibility, resolves it against concurrent bids, updates the price, and broadcasts the change to all connected watchers. The bid screen discloses the flat convenience fee that will be added if the bid wins.
- An outbid buyer receives a push alert and counter-bids within the same session.
- The fraud service identifies a shill-bidding pattern, flags the auction, opens a dispute case, and blocks settlement pending administrator review.
- An auction closes; the orchestrator generates an invoice for the winning bid plus the flat convenience fee, captures the total, and holds it in platform escrow. The seller is notified to ship.
- The buyer confirms receipt, which releases the escrowed amount to the seller immediately, net of the fee.
- The buyer instead raises a dispute inside the protection window, attaching evidence. The escrowed funds are frozen, an administrator adjudicates, and the outcome is either a refund to the buyer or a release to the seller.
- The protection window expires with no dispute and no confirmation. A scheduled job releases the funds to the seller automatically, and the dispute option is permanently closed for that order.

5. Scope & High-Level Features

Total estimated effort: 1600 person-hours (200 hours per member). Component-level unit testing is budgeted inside each feature; the testing line below covers cross-service integration, end-to-end, and load testing only.

Feature	Description	Hours
Frontend (React)	Bidding UI, live price and fee disclosure, KYC upload flow, dispute form, notification tray, responsive layout	190
Payment, Fees & Escrow	Razorpay (test mode), flat convenience-fee engine, escrow hold and release, Saga compensation, refunds	180
Bid Processing	Validation, deterministic conflict resolution, idempotency keys, Redis buffering, batched writes	170

continued on next page

Feature	Description	Hours
Fraud Detection	Bid-event schema, labelled synthetic dataset, shill-bid detection, device fingerprinting, bid-ring clustering	150
Auction Orchestration	Lifecycle state machine, anti-snipe auto-extension, settlement and escrow-hold transitions	115
Infrastructure	Docker, Docker Compose, Render blueprint, Vercel deployment, scheduled-job runner, CI/CD, Prometheus/Grafana, Jaeger	115
Integration Testing	Contract tests, end-to-end scenarios, ledger reconciliation, load and soak testing	110
Dispute Management	Dispute lifecycle, evidence capture, window enforcement, auto-release scheduler, admin adjudication, refund execution	105
Documentation	OpenAPI spec, architecture decision records, settlement policy, user manual	95
Pricing Engine	Bid-velocity tracking, dynamic increment calculation, hot-auction flagging	85
KYC & Seller Verification	Document submission, encrypted document store, verification state machine, listing gate, admin review queue	75
Admin & Moderation	Listing review, user suspension, audit log	60
Search & Analytics	Full-text search, filtering, seller dashboard, trend reporting	55
Auth & User Service	Registration, JWT issuance, roles, profiles, session management	50
WebSocket Gateway	Connection management, room fan-out, outbid alerts, reconnect handling	45
Total		**1600**

Settlement policy (fixed rules the implementation must enforce).

- **Convenience fee:** a single configured flat amount per completed sale, not a percentage of the hammer price. It is disclosed on the bid screen before a bid is placed, added to the winning bid at invoice generation, captured with the total, and retained by the platform on release. It is non-refundable on a buyer-fault dispute outcome and refunded in full when the platform or seller is at fault. Losing bidders are never charged.
- **Escrow hold:** the captured total is held by the platform, not forwarded to the seller, from capture until release. No code path transfers funds to a seller outside the release transition.
- **Protection window:** a configurable duration (default 7 days) beginning when the seller marks the item shipped. A dispute may be raised only while the window is open; the API rejects a dispute request on a closed window.
- **Release triggers:** buyer confirms receipt (immediate release); window expires with no dispute (scheduled automatic release); or administrator adjudication resolves an open dispute in the seller's favour.
- **Refund trigger:** administrator adjudication resolves an open dispute in the buyer's favour, reversing the capture through the payment provider.
- **Terminal state:** once released or refunded, the order is closed and no dispute may be opened against it.

6. Technology Stack

Layer	Technology	Rationale
Backend	Python 3.11 + FastAPI	Native async I/O, automatic OpenAPI generation
Messaging	Apache Kafka	Ordered, replayable event log; one stream feeds both bidding and fraud detection
Database	PostgreSQL 15	ACID guarantees for money, escrow ledger, and bid history
Cache/Buffer	Redis 7	In-memory current-price reads and write buffering
Real-Time	FastAPI WebSockets	Native to the backend framework; no extra protocol layer
Frontend	React 18 + Tailwind CSS	Component model suits frequently updating price views
Auth	JWT (access + refresh)	Stateless verification across services; verification status carried as a claim
Document store	S3-compatible object storage, encrypted at rest	KYC documents must not sit on an ephemeral container filesystem; access is signed-URL only and time-limited
Scheduling	Render Cron Job	Periodic sweep that releases escrow on expired protection windows
Monitoring	Prometheus + Grafana + Jaeger	Metrics, dashboards, and distributed tracing
Containers	Docker + Docker Compose	One image per service; a single Compose file reproduces the whole system locally and in CI
Backend host	Render	Each service deploys independently from its Dockerfile; managed Postgres and Key Value (Redis-compatible) remove datastore operations. Kafka runs as a Render private service with a persistent disk, since Render has no managed Kafka offering
Frontend host	Vercel	Static SPA build on a global CDN with a preview deployment per pull request
CI/CD	GitHub Actions	Lint, test, and image build on every PR; deploy hooks to Render and Vercel on merge to main
Payments	Razorpay (test mode)	Route transfers support a settlement hold with timed release and reversal, which map directly onto the escrow window; INR-native; no real funds handled

7. Third-Party APIs

The platform integrates the following third-party APIs:

- Razorpay
- Firebase Cloud Messaging (push notifications)
- Cloudinary
- Prometheus

Any additional third-party APIs required by the implementation will be identified and specified in subsequent design documents.

8. Roles & Responsibilities

Each member is allocated exactly 200 hours. Per-member row totals and per-component sums both reconcile with the feature table in Section 5.

Member	ID	Components (hours)	Total
Naitik	202512006	Bid Processing — validation and conflict resolution (115); Auction Orchestration — lifecycle transitions (30); Dispute Management — window timer and auto-release scheduler (25); Integration testing (15); Documentation (15)	200
Sumit	202512124	Bid Processing — Redis buffering, idempotency, Kafka producer (55); Auction Orchestration — state machine, auto-extend, settlement and escrow-hold transitions (85); Payment — invoice generation with convenience fee (25); Integration testing (20); Documentation (15)	200
Sagar	202512107	Pricing Engine (85); KYC & Seller Verification — submission, document store, verification state machine, listing gate (55); Search & Analytics — indexing and query layer (30); Integration testing (15); Documentation (15)	200
Harshal	202512013	Fraud Detection — bid-event schema and labelled synthetic dataset generator, weeks 1–3 (30); shill detection, fingerprinting, bid-ring clustering (120); Dispute Management — fraud-triggered dispute creation and evidence linkage (20); Integration testing (15); Documentation (15)	200
Jaydip	202512090	Payment, Fees & Escrow — Razorpay, convenience-fee engine, escrow hold/release, Saga coordination, webhooks (155); Dispute Management — refund execution and ledger reconciliation (25); Integration testing (10); Documentation (10)	200
Gaurang	202512007	Frontend — bidding UI, live price and fee disclosure, KYC upload flow, dispute form, notifications (115); Frontend — WebSocket client integration (30); Search & Analytics — seller dashboard (25); Integration testing (15); Documentation (15)	200
Jay	202512123	Admin & Moderation — listing review, suspension, audit log (60); Dispute Management — admin adjudication console (35); KYC — admin verification review queue (20); Frontend — shared components, responsive layout (45); Auth & User Service — login and profile screens (20); Integration testing (10); Documentation (10)	200

continued on next page

Member	ID	Components (hours)	Total
Ayush	202512084	Infrastructure — Docker/Compose, Render blueprint, Vercel deployment, object storage, scheduled-job runner, CI/CD, observability (115); WebSocket gateway — server side (45); Auth & User Service — JWT, gateway middleware, role and verification gating (30); Integration and load testing (10)	200
Team total			**1600**

Component ownership. Where a component is split across members, one member owns the interface and merges the work: Naitik owns Bid Processing, Sumit owns Auction Orchestration, Jaydip owns Payment & Escrow, Sagar owns KYC & Seller Verification, Jay owns Dispute Management, Gaurang owns the Frontend, and Ayush owns Auth & User Service and the WebSocket Gateway. Contributors work against the owner's published contract.

Shared obligations (included in the hours above): every member reviews 2–3 pull requests per week, writes unit tests for their own components to the 80% coverage threshold, and attends the weekly standup. No pull request merges without two approvals.

9. Timeline & Milestones

Sixteen weeks, beginning Monday 3 August 2026. Phases mark *integration milestones*, not exclusive work windows: each member works their allocated components continuously at roughly 12.5 hours per week, and a phase row names what must be integrated and demonstrable by the end of that window.

Weeks	Phase	Integration milestone	Dates
1–2	Setup	Repository, Docker Compose environment, architecture document, database schema, bid-event schema, settlement policy fixed (fee amount, window length), CI skeleton	Aug 3 – Aug 14
3–5	Bid Service MVP	Bid validation, conflict resolution, Kafka topics, Redis buffer, auth service, labelled fraud dataset generator complete	Aug 17 – Sep 4
6–7	Identity & Real-Time	Seller KYC submission, admin approval, listing gate enforced; auction state machine, auto-extension, WebSocket gateway, live price push	Sep 7 – Sep 18
8–9	Pricing, Payments & Fees	Pricing engine, Razorpay integration, convenience-fee calculation, escrow hold, Saga compensation paths, baseline load measurement	Sep 21 – Oct 2
10	Fraud Detection v1	Streaming shill detection scored against the week-3 dataset, dispute record creation, demo preparation	Oct 5 – Oct 9
11	**Mid-Semester Demo**	**End-to-end auction: KYC-gated listing, bid, settle with fee into escrow. Code walkthrough.**	**Oct 12**
12–13	Disputes, Search & Admin	Dispute lifecycle and window enforcement, auto-release scheduler, adjudication console, refunds, full-text search, seller dashboard, fraud tuning	Oct 19 – Oct 30

continued on next page

Weeks	Phase	Integration milestone	Dates
14–15	Testing & Deployment	End-to-end suite, ledger reconciliation, load testing, Render and Vercel deployment, monitoring dashboards	Nov 2 – Nov 13
16	**Final Demo**	**Live deployment on Render and Vercel, measured performance results, final presentation**	**Nov 18**

Checkpoints: 15-minute standup every Tuesday; sprint board updated continuously; a written progress note at the end of weeks 5, 9, and 13 recording actual hours logged against the 200-hour allocation, so drift is caught early rather than at the end.

10. Success Metrics

Load and latency figures are measured against the containerised deployment on a dedicated 8 vCPU / 16 GB host, not on developer laptops. Functional, resilience, and availability metrics are verified on the live Render and Vercel deployment.

- **Throughput:** sustain 1000 bids per second aggregate across 50 concurrent auctions for 10 minutes with an error rate no higher than 0.1%, and 200 bids per second sustained against a *single* auction, which is the contended case Section 3 identifies.
- **Latency:** 95th-percentile bid acknowledgement under 500 ms; 95th-percentile WebSocket fan-out from bid acceptance to watcher receipt under 100 ms, measured with 500 concurrent watchers on a single auction.
- **Settlement correctness:** over 1000 simulated settlements, every order reconciles exactly — captured amount equals winning bid plus the flat fee, the seller payout equals the captured amount minus the fee, and no order is both released and refunded. Zero tolerance: a single reconciliation failure fails the metric.
- **Escrow and window enforcement:** no funds release before the protection window closes or the buyer confirms; a dispute request against a closed window is rejected; the scheduled sweep releases expired holds within 5 minutes of expiry. Verified by a time-accelerated test fixture covering every terminal path.
- **KYC gate:** zero auctions created by an account not in the verified state, tested directly against the service API rather than through the UI.
- **Resilience:** stopping any one backend service leaves the remaining functionality available, and the service returns without manual intervention via its container restart policy and Render health check. Verified by a documented chaos test per service.
- **Fraud detection:** on a seeded dataset of 500 synthetic auctions containing 200 labelled shill-bidding rings, achieve recall ≥ 0.90 at precision ≥ 0.75, with alerts raised before auction close.
- **Code quality:** 80% unit-test coverage on backend services, zero linting errors in CI, two approvals required per merge.
- **Effort:** each of the 8 members logs 200 hours; team total 1600 hours, tracked weekly.

11. Risks & Mitigation

Risk	Impact	Mitigation
Holding buyer funds is a regulated activity	High	The platform never touches real money. Razorpay test mode only, synthetic accounts, and the escrow ledger is an internal record of a simulated hold. This is stated in the user manual and in an architecture decision record, and the system is presented as a functional simulation of escrow, not a payment service
KYC documents are sensitive personal data	High	Only synthetic or team-member-consented sample documents are used. Documents are stored in object storage encrypted at rest, retrieved by time-limited signed URL, never logged, and purged at project end. Retention and lawful-basis notes are recorded in an ADR
Kafka operational complexity exceeds team experience	High	Single-broker Compose setup through week 13; broker count revisited only if week 14 load testing shows it is the bottleneck
Escrow and dispute states expand the settlement state machine	Medium	Every transition is specified as a table in week 1 before implementation, and the time-accelerated test fixture in Section 10 exercises each terminal path. Money-moving transitions are guarded by idempotency keys so a retried webhook cannot double-release
Render provides no managed Kafka	Medium	Kafka runs as a Render private service with a persistent disk, provisioned and smoke-tested in week 2 rather than at deployment time; an external managed cluster is the documented fallback
Free-tier instances idle out or throttle under load	Medium	Load figures are measured on a dedicated host, not on Render; the hosted deployment demonstrates functional correctness and resilience. Paid instance tier budgeted for weeks 10–11 and 14–16 so services do not spin down mid-demo, and the escrow sweep runs as a cron job rather than an in-process timer
Razorpay test-mode restrictions on escrow-style flows	Medium	Settlement hold, timed release, and reversal cover the required primitives. The payment service sits behind an interface with a mock implementation; Razorpay is one adapter, not a dependency of the demo
Performance targets in Section 10 not met	Medium	Baseline measurement in week 9, six weeks before the final demo, so the gap is known early. If a target is missed, the measured curve and the identified bottleneck are reported rather than the target being quietly dropped
Fraud detection accuracy target not met	Medium	Labelled synthetic dataset built in weeks 1–3, before any detector exists, so the metric is measurable from the first iteration and tuning has a fixed target
Uneven contribution across members	Medium	Hours logged per member and reviewed at the week 5, 9, and 13 checkpoints; work reassigned if variance exceeds 15%

12. Conclusion

ZeoAuction exercises the core concerns of service-oriented design: decomposition into independently deployable services, asynchronous event-driven integration, consistency across transaction boundaries without distributed locking, real-time stream processing, and fault isolation. The 1600 person-hour scope divides into eight equal 200-hour allocations, each anchored to a named component with its own deliverable, so individual contribution is both substantial and separately assessable. The technical difficulty sits where it should: in deterministic conflict resolution under concurrency, in detecting adversarial behaviour from a live stream fast enough to act on it, and in holding and settling money correctly across service boundaries — through a verification gate, a fee, an escrow hold, and a time-bounded dispute window — when any participant may fail mid-transaction. Docker keeps the local, CI, and hosted environments identical, Render and Vercel remove deployment from the critical path, and Prometheus, Grafana, and Jaeger make the distributed behaviour observable enough to evaluate against the metrics in Section 10.